

Inheritance

Q1.

Create a Java class called Song with properties title and artist, then create a subclass PremiumSong that adds an extra property lyricsAccess (boolean) to indicate if the user can view lyrics.

Answer:

```
class Song {
    String title;
    String artist;

    public Song(String title, String artist) {
        this.title = title;
        this.artist = artist;
    }

    void display() {
        System.out.println("Title: " + title + ", Artist: " + artist);
    }
}

class PremiumSong extends Song {
    boolean lyricsAccess;

    public PremiumSong(String title, String artist, boolean lyricsAccess) {
        super(title, artist);
        this.lyricsAccess = lyricsAccess;
    }

    @Override
    void display() {
        super.display();
        System.out.println("Lyrics Access: " + lyricsAccess);
    }
}

public class SongDemo {
    public static void main(String[] args) {
        Song regular = new Song("Perfect", "Ed Sheeran");
        PremiumSong premium = new PremiumSong("Shape of You", "Ed Sheeran", true);

        regular.display();
        premium.display();
    }
}

/* Sample Output:
Title: Perfect, Artist: Ed Sheeran
Title: Shape of You, Artist: Ed Sheeran
Lyrics Access: true
*/
```

Q2.

Build a class `FoodOrder` with a method `calculateTotal()` that returns the order amount, then extend it with a `ZomatoGoldOrder` subclass that overrides `calculateTotal()` to apply a 10% discount for Gold users. Hint: Use the `super` keyword to call the parent class's `calculateTotal()` method before applying the discount.

Answer:

```
class FoodOrder {
    double amount;

    public FoodOrder(double amount) {
        this.amount = amount;
    }

    double calculateTotal() {
        return amount;
    }
}

class ZomatoGoldOrder extends FoodOrder {
    public ZomatoGoldOrder(double amount) {
        super(amount);
    }

    @Override
    double calculateTotal() {
        double baseTotal = super.calculateTotal(); // get original amount
        double discount = baseTotal * 0.10;
        return baseTotal - discount;               // apply 10% Gold discount
    }
}

public class FoodOrderDemo {
    public static void main(String[] args) {
        FoodOrder normalOrder = new FoodOrder(500);
        ZomatoGoldOrder goldOrder = new ZomatoGoldOrder(500);

        System.out.println("Normal order total: Rs. " + normalOrder.calculateTotal());
        System.out.println("Zomato Gold order total: Rs. " + goldOrder.calculateTotal());
    }
}

/* Sample Output:
Normal order total: Rs. 500.0
Zomato Gold order total: Rs. 450.0
*/
```

Q3.

Write a class `UserProfile` with a method `displayProfile()` that prints the user's name. Create two subclasses: `InfluencerProfile` (adds `followers` property) and `BrandProfile` (adds `brandName` property). Demonstrate hierarchical inheritance by creating objects of both subclasses and calling `displayProfile()`.

Answer:

```
class UserProfile {
    String name;

    public UserProfile(String name) {
        this.name = name;
    }
}
```

```

        void displayProfile() {
            System.out.println("Name: " + name);
        }
    }

// Hierarchical inheritance: both subclasses extend the same parent class
class InfluencerProfile extends UserProfile {
    int followers;

    public InfluencerProfile(String name, int followers) {
        super(name);
        this.followers = followers;
    }

    @Override
    void displayProfile() {
        super.displayProfile();
        System.out.println("Followers: " + followers);
    }
}

class BrandProfile extends UserProfile {
    String brandName;

    public BrandProfile(String name, String brandName) {
        super(name);
        this.brandName = brandName;
    }

    @Override
    void displayProfile() {
        super.displayProfile();
        System.out.println("Brand Name: " + brandName);
    }
}

public class HierarchicalDemo {
    public static void main(String[] args) {
        InfluencerProfile influencer = new InfluencerProfile("Aditi Rao", 85000);
        BrandProfile brand = new BrandProfile("Marketing Team", "Nykaa");

        influencer.displayProfile();
        System.out.println("---");
        brand.displayProfile();
    }
}

/* Sample Output:
Name: Aditi Rao
Followers: 85000
---
Name: Marketing Team
Brand Name: Nykaa
*/

```

Q4.

Create three classes: AppUser (base), VerifiedUser (extends AppUser), and CelebrityUser (extends VerifiedUser). In CelebrityUser, use the super keyword to call the parent constructor and print a welcome message including the user's name. Constraint: Demonstrate multilevel inheritance and constructor chaining using super().

Answer:

```
class AppUser {
    String name;

    public AppUser(String name) {
        this.name = name;
        System.out.println("AppUser constructor called for: " + name);
    }
}

class VerifiedUser extends AppUser {
    public VerifiedUser(String name) {
        super(name);
        System.out.println("VerifiedUser constructor called - badge granted.");
    }
}

class CelebrityUser extends VerifiedUser {
    public CelebrityUser(String name) {
        super(name); // calls VerifiedUser's constructor, which calls AppUser's
        System.out.println("Welcome, " + name + "! You are a verified celebrity user.");
    }
}

public class MultilevelDemo {
    public static void main(String[] args) {
        CelebrityUser celeb = new CelebrityUser("Virat Kohli");
    }
}

/* Sample Output:
AppUser constructor called for: Virat Kohli
VerifiedUser constructor called - badge granted.
Welcome, Virat Kohli! You are a verified celebrity user.
*/
// This demonstrates multilevel inheritance: AppUser -> VerifiedUser -> CelebrityUser,
// with each constructor chaining to its parent using super().
```

Q5.

Refactor the following code so that it uses the `this` keyword to clearly distinguish between class properties and method parameters: `class Product { String name; Product(String name) { name = name; } }` Hint: The constructor should assign the parameter to the class property correctly.

Answer:

```
// Original buggy code:
// class Product {
//     String name;
//     Product(String name) {
//         name = name; // BUG: assigns parameter to itself, class field stays null
//     }
// }

// Fixed version using 'this' keyword:
class Product {
    String name;

    Product(String name) {
        this.name = name; // 'this.name' refers to the class field,
                          // 'name' (right side) refers to the constructor parameter
    }
}
```

```
}

void display() {
    System.out.println("Product Name: " + this.name);
}

public static void main(String[] args) {
    Product product = new Product("Running Shoes");
    product.display();
}
}
```

/* Sample Output:

Product Name: Running Shoes

*/

// Without 'this', Java cannot tell the field 'name' apart from the parameter 'name'

// since they share the same identifier - it simply assigns the parameter to itself.